
PART IV — FROM RAG TO AGENTIC RAG

Chapter 16

Designing the Agent Loop

“Freedom without state becomes repetition; freedom without limits becomes a runaway loop.”

Chapter 16 — Designing the Agent Loop

Chapter 15 defined agency as bounded runtime choice. We now need to turn that idea into control flow. An agent loop is the smallest structure that lets a model observe the current run, choose an allowed action, receive the result, and decide again. The difficult part is not writing a `while` statement. It is designing state, transitions, budgets, and termination rules, so the loop remains useful when the model behaves imperfectly.

Memora provides unusually concrete evidence for this design. Its first free-form tool loop allowed the model too much procedural freedom; the model sometimes judged an answer before retrieving, batched incompatible actions, or continued for more than forty iterations. The project responded with a four-phase RETRIEVE → COMPRESS → DRAFT → JUDGE state machine, explicit counters, query deduplication, and enforced protocol ordering. By the end of this chapter, you will be able to sketch a minimal loop, choose what belongs in short-term state, and explain why every exit condition is part of correctness—not merely cost control.

16.1 The reasoning loop: decide → retrieve → evaluate → answer

Definition — Agent loop

A bounded control cycle in which the system repeatedly inspects run state, selects an allowed action, executes it, records the result, and either continues or terminates according to explicit quality and resource conditions.

A useful loop separates four responsibilities. `Decide` chooses the next permitted action. `Retrieve` executes an external search and returns evidence. `Evaluate` asks whether the evidence covers the user's needs and whether another action is justified. `Answer` synthesizes only after the system has either reached sufficient quality or exhausted a deliberate fallback policy.

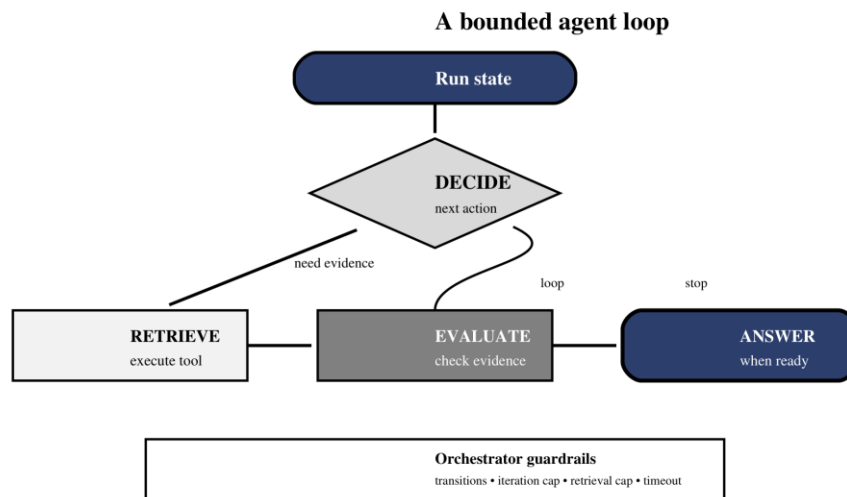


Figure 16.1 — The decision loop is adaptive inside deterministic guardrails.

Figure 16.1 shows two kinds of control at once. The inner path is adaptive: the current state may lead to retrieval, evaluation, another retrieval, or an answer. The outer boundary is deterministic: only known actions are allowed, and budgets always apply. This separation is the heart of robust agent design. The model contributes semantic judgment; ordinary program logic owns invariants.

Analogy — A laboratory investigation

A careful scientist does not write the conclusion before collecting evidence. They form a question, run a test, inspect the result, and decide whether another test is needed. The laboratory protocol restricts which tests are safe and when the investigation must stop. The scientist supplies judgment; the protocol supplies discipline.

Memora’s imperative implementation refined the generic loop into four explicit phases. RETRIEVE accumulates evidence, COMPRESS removes noise and redundancy, DRAFT produces a candidate answer without tool distractions, and JUDGE evaluates coverage and grounding. An insufficient verdict returns the run to retrieval while preserving useful evidence. A satisfactory verdict permits the final answer. This phase design exists because small models did not reliably remember procedural rules as context grew.

A complete phase walk-through

Consider the query, “Compare the safety features of an adjustable speed drive and explain which hazards each feature mitigates.” RETRIEVE first generates focused variants for overcurrent protection, thermal protection, emergency stopping, and fault isolation. Each result is validated before it enters an accumulated state. Evidence that merely mentions a product catalogue without explaining a safety mechanism is dropped rather than carried forward.

COMPRESS then operates on the accumulated evidence rather than the last retrieval alone. Neighbor-aware merging joins adjacent fragments from the same source; deduplication removes repeated statements; relevance compression keeps sentences that support the requested feature-to-hazard mapping. DRAFT receives a clean, stateless package containing the question and compressed evidence, which avoids sending the tool-call history that previously caused a local inference server to return HTTP 500.

JUDGE checks more than prose quality. It asks whether every requested feature is supported, whether the answer maps feature to hazards, and whether claims remain inside the retrieved context. An ‘INSUFFICIENT’ verdict should name what is missing—perhaps emergency-stop behavior—so the next RETRIEVE phase has a positive target. A bare “try again” wastes the feedback signal and encourages near-duplicate queries.

16.2 Why tool-calling is the core primitive

Definition — Tool call

A structured request emitted by a model that names an application-provided operation and supplies arguments matching a declared schema. The application—not the model—executes the operation and returns its results.

Tool-calling gives the model a narrow vocabulary of actions. Instead of asking the model to describe how it would search, the application exposes a ‘retrieve_documents’ operation. Instead of allowing arbitrary code, the schema accepts only a query string. This turns natural-language intent into a message the orchestrator can validate, log, execute, reject, or retry.

```

retrieve_tool = {
  "name": "retrieve_documents",
  "description": "Search approved knowledge sources.",
  "parameters": {
    "type": "object",
    "properties": {
      "query": {"type": "string", "minLength": 3}
    },
    "required": ["query"],
    "additionalProperties": False,
  },
}

```

The schema is a contract, not a security boundary by itself. The orchestrator still validates the arguments and injects server-owned dependencies such as the retriever, collection names, and configured retrieval depth. Memora deliberately removed `top\_k` from the LLM-visible schema because an 8B model selected inconsistent values. `RETRIEVAL\_TOP\_K` and `RETRIEVAL\_TOP\_L` became configuration, while the model retained the decision it was good at: choosing the search wording.

Concern	Model should decide	Program should enforce
Search intent	Query wording and missing aspect	Allowed sources and argument validation
Retrieval depth	Usually no	Configured `top_k` / `top_l` and score thresholds
Sequencing	Propose the next action	Retrieval before compression; validation before acceptance
Execution	Never execute directly	Call the retriever and return a bounded result
Termination	May signal readiness	Hard caps, timeouts, and final fallback

Common pitfall — Letting the model own the procedure

Memora's earlier free-form loop exposed too many actions at once. The model sometimes called the quality check before retrieval or attempted incompatible actions in one turn. The fix was to restrict tools by phase and make quality checking a direct application call.

16.3 Iteration caps and termination conditions

Every loop needs more than one stop condition because different failures consume different resources. An iteration cap limits reasoning turns. A total-retrieval cap limits external searches across the whole run. A per-iteration tool-call cap limits bursts within one model response. A timeout limits wall-clock exposure. A quality condition stops successful work early. Together, these rules create a bounded search process.

Guard	What it prevents	Memora lesson
`MAX_ITERATIONS`	Endless decide/act cycles	Added after a run exceeded 40 iterations
`MAX_TOTAL_RETRIEVALS`	Unbounded evidence accumulation	Stops repeated variants from exploding context
`MAX_TOOL_CALLS_PER_ITERATION`	One response requesting too many actions	Must preserve tool-call/result pairing when truncating
Query deduplication	Repeating the same failed search	A `seen_queries` set blocks exact repeats
Quality acceptance	Continuing after the task is complete	A grounded, relevant verdict enables answer return
Timeout / provider abort	Waiting indefinitely on external services	Retry logic eventually surfaces a controlled failure

The numbers are operating parameters, not universal truths. Memora used values such as six iterations and five total retrievals in its imperative loop because dry runs showed that they bound cost while leaving room for reformulation. The later LangGraph workflow replaced some counter-based limits with structural routes and retry limits. What remains invariant is the requirement that every cycle consumes a measurable budget.

Failure analysis — the forty-iteration loop

Memora’s runaway-loop bug is a concrete lesson in missing invariants. With no iteration cap, no per-turn query deduplication, and no context pruning, the model generated increasingly similar retrievals for more than forty iterations. Tool results accumulated in message history until the request exceeded the provider’s token-per-minute limit. The eventual HTTP error was only visible symptom; the architectural failure occurred much earlier, when the program allowed a cycle that did not demonstrate progress.

The repair combined several controls because no single cap addresses every failure. `MAX\_ITERATIONS` bounded reasoning turns. `MAX\_TOTAL\_RETRIEVALS` bounded searches across the run. `MAX\_TOOL\_CALLS\_PER\_ITERATION` bounded bursts. A `seen\_queries` set blocked exact repeats. Compression replaced bulky raw results with a smaller context while preserving tool-call/result pairing. Together, these controls converted an open-ended conversation into a finite state transition system.

Analogy — A fuel gauge and a progress map

A car trip needs both fuel limits and evidence that the route is approaching the destination. A retrieval budget is the fuel gauge; coverage improvement is the progress map. Either one alone is insufficient: unlimited fuel permits wandering, while a strict fuel cap without progress checks may stop a trip that was one useful turn from completion.

```
def can_continue(state, limits):  
    return (  
        not state.quality_satisfied  
        and state.iterations < limits.max_iterations  
        and state.total_retrievals < limits.max_retrievals  
        and state.elapsed_seconds < limits.timeout_seconds  
    )
```

Common pitfall — Slicing tool calls without repairing history

If the assistant emits seven tool calls, but the program executes only five, the conversation still contains two unmatched call IDs unless it is repaired. Either remove the skipped calls from the assistant message or append explicit skipped results. A cap that corrupts protocol history is not a safe cap.

16.4 Short-term memory within a session

Definition — Working memory

The ephemeral state used during one agent run: user intent, attempted queries, retrieved evidence, intermediate drafts, quality verdicts, and counters. It is distinct from source documents and from persistent, learned memory.

Without state, each iteration is amnesiac. The agent cannot know which queries it already tried, which chunks survived validation, or why the previous draft was rejected. With an undisciplined state, every raw tool result remains in the conversation until the context window becomes a new failure. Good state design records what the next decision needs while compressing or discarding what it does not.

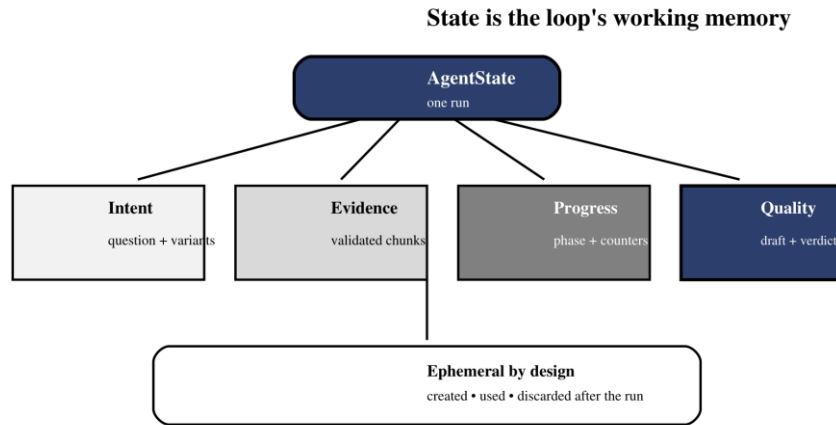


Figure 16.2 — Short-term state carries intent, evidence, progress, and quality through one run.

Figure 16.2 separates state by purpose so that a trace can answer four different questions: what the user meant, what evidence was found, how much budget remains, and why the latest draft passed or failed. Combining all four into an unstructured transcript makes stop decisions and tests far harder.

16.4.1 State invariants that keep the loop honest

A state object needs invariants, not just fields. The iteration counter must increase exactly once per cycle. Retrieval count cannot exceed its configured maximum. Compression may run only after evidence exists. A draft cannot be marked accepted without a recorded judge's outcome, and a terminal state cannot transition back into retrieval. These properties should be asserted at node boundaries, so a wiring error fails near its source.

```
def assert_state(state):
    assert 0 <= state.iteration <= state.max_iterations
    assert state.retrievals <= state.max_retrievals
    if state.phase == "DRAFT":
        assert state.compressed_context
    if state.status == "accepted":
        assert state.quality_score >= state.required_score
    if state.is_terminal:
        assert state.next_action is None
```

These checks also improve replay. Given the same initial state and recorded model/tool outputs, the orchestrator should reproduce the same transitions and terminal reason. The language model remains nondeterministic, but the application interpretation of recorded outputs does not need to be. Deterministic transition logic is what turns a long trace into an explainable run rather than a mystery transcript.

Memora separates evidence into document and learned-QA tracks; records variants tried, accumulates chunks that survive validation, and stores the current phase, draft, verdicts, and counters. The state is created for a request and disappears after the request unless selected information is deliberately written to the feedback or learning layer. That boundary prevents temporary reasoning debris from masquerading as durable knowledge.

- Store identifiers and compact summaries when full raw payloads are unnecessary.
- Keep server dependencies and feature flags outside run state; inject them as configuration.
- Use typed fields so every node knows what it may read and write.

- Preserve provenance with each evidence item so the final answer can cite what survived.
- Track counters explicitly; never ask the model to count turns by rereading chat history.

16.5 The minimal agent in pseudocode

A minimal agent does not need a framework. It needs a typed state, a small action set, a validated tool boundary, and a loop whose success and failure exits are explicit. The following skeleton intentionally leaves retrieval and judging behind interfaces, so the control logic remains testable.

```
def run_agent(question, services, limits):
    state = AgentState(question=question)

    while can_continue(state, limits):
        state.iterations += 1
        action = choose_action(state, allowed=["retrieve", "answer"])

        if action.name == "retrieve":
            query = validate_query(action.arguments["query"])
            if query in state.queries_tried:
                state.notes.append("duplicate query rejected")
                continue

            state.queries_tried.add(query)
            result = services.retriever.search(query)
            state.total_retrievals += 1
            state.evidence = validate_and_merge(state.evidence, result)
            state.quality_satisfied = evidence_is_sufficient(
                state.question, state.evidence
            )
            continue

        if action.name == "answer" and state.quality_satisfied:
            return generate_grounded_answer(state.question, state.evidence)

    return safe_fallback(state)
```

Test the loop with adversarial paths, not only the happy path: empty retrieval, duplicate queries, malformed tool arguments, too many calls, a judge that never accepts, and a provider timeout. Each test should prove that state remains coherent and that the run exits predictably. In Memora, the runaway-loop bug was not a model of curiosity; it was evidence that termination had not yet been designed as part of the architecture.

A deterministic test harness for a probabilistic loop

The LLM may be probabilistic, but the orchestrator can still be tested deterministically. Replace the model with a scripted fake that emits a known sequence of actions and replace retrieval with fixtures. One test emits a duplicate query; another emits seven tool calls; another returns insufficient quality forever. Assertions should inspect final state, executed calls, exit reason, and message pairing—not only the final answer string.


```
def test_duplicate_query_does_not_consume_retrieval_budget():
    model = ScriptedModel([
        retrieve_call("motor overload protection"),
        retrieve_call("motor overload protection"),
        answer_call(),
    ])
    retriever = FixtureRetriever({
        "motor overload protection": [OVERLOAD_CHUNK]
    })

    result = run_agent("How is overload handled?", model, retriever)

    assert retriever.calls == ["motor overload protection"]
    assert result.state.total_retrievals == 1
    assert result.exit_reason in {"quality_satisfied", "safe_fallback"}
```

Test path	Invariant to assert	Expected exit
No chunks returned	No fabricated evidence enters state	No-context answer
Duplicate variant	Budget is not consumed twice	Continue or safe fallback
Judge always insufficient	Iteration/retry cap is honored	Budget exhausted
Too many tool calls	Every retained call has a paired result	Continue with repaired history
Provider timeout	State remains serializable and coherent	Controlled service failure